

Laboratorio 3: Publish & Subscribe

Sistema Event-Driven para Ingesta y Agregación de Eventos

Departamento de Ingeniería Informática

Universidad de Santiago de Chile

Prof. Miguel Cárcamo

Semestre 2-2025

1. Objetivos Pedagógicos

Al finalizar este laboratorio, los estudiantes serán capaces de:

- Diseñar e implementar sistemas event-driven usando patrones publish-subscribe
- Comprender y aplicar semánticas de entrega (at-least-once) en sistemas distribuidos
- Implementar mecanismos de idempotencia y deduplicación de eventos
- Manejar eventos duplicados y fuera de orden (out-of-order) en pipelines de procesamiento
- Diseñar estrategias de backpressure y retries con políticas definidas
- Implementar tolerancia a fallas en sistemas pub-sub (recuperación de consumidores y brokers)
- Demostrar capacidades de replay y re-procesamiento según la tecnología elegida
- Implementar observabilidad mínima: logs estructurados y métricas (throughput, error rate, lag/backlog)
- Comparar diferentes tecnologías pub-sub (Kafka, RabbitMQ, MQTT, NATS JetStream, Redis Streams) resolviendo el mismo problema
- Aplicar buenas prácticas de DevOps: contenedores, CI/CD, y reproducibilidad

2. Contexto del Curso y Logística

2.1. Organización

- **Número de estudiantes:** 20 estudiantes
- **Formación de grupos:** 5 grupos de 4 integrantes
- **Duración del laboratorio:** Aproximadamente 1 mes (4 semanas)
- **Modalidad:** Cada grupo implementa el mismo problema usando una tecnología diferente

2.2. Elección de Tecnologías

Cada grupo debe elegir una de las siguientes tecnologías pub-sub:

1. **Kafka o Redpanda** (Kafka-compatible)
2. **RabbitMQ**
3. **MQTT** (Mosquitto)
4. **NATS JetStream**
5. **Redis Streams**

Importante:

- Cada grupo debe comunicar su elección de tecnología al profesor lo antes posible a través de Google Classroom.
- Se recomienda que los grupos no repitan tecnologías para permitir comparación entre diferentes soluciones.
- Todas las tecnologías deben resolver el mismo problema y cumplir los mismos requisitos funcionales para permitir comparación entre grupos.

2.3. Lenguaje de Programación

- **Sugerido:** Python (por simplicidad y ecosistema)
- **Alternativas permitidas:** Go, Java (si el grupo lo prefiere)
- **Requisito:** El lenguaje debe tener clientes maduros para la tecnología pub-sub elegida

2.4. Entrega

- **Plataforma:** Repositorio Git (GitLab o GitHub)
- **Requisito:** CI/CD configurado (al menos compilación/validación básica)
- **Formato:** Repositorio público o privado con acceso para el profesor

3. Descripción del Problema

3.1. Contexto del Sistema

Se requiere desarrollar un **sistema event-driven** para el procesamiento de eventos de un país ficticio (denominado “País X”). El sistema debe manejar datos sintéticos y neutrales, sin referencias a países o situaciones reales.

Propósito del sistema: El sistema está diseñado para dar cuenta del estado de delincuencia y seguridad de diferentes zonas o regiones de un país. A través de la recolección, procesamiento y agregación de información sobre delincuencia, victimización y migración,

el sistema permite generar métricas y análisis que contribuyen a la comprensión de los patrones de seguridad en diferentes áreas geográficas.

El sistema generará y procesará tres tipos de eventos de forma aleatoria y continuamente:

- **Delitos** (`security.incident`): Eventos relacionados con actos delictuales reportados en diferentes regiones
- **Encuestas de victimización** (`survey.victimization`): Respuestas de encuestas sobre experiencias de victimización de la población
- **Casos de migración** (`migration.case`): Casos relacionados con procesos migratorios que pueden estar asociados a factores de seguridad

Estos eventos, una vez procesados y agregados, permiten generar métricas diarias por región que reflejan el estado de seguridad y delincuencia de cada zona, facilitando la toma de decisiones y el análisis de tendencias.

3.2. Flujo del Sistema

El sistema debe implementar el siguiente pipeline:

1. **Generación/Publicación:** Un generador crea eventos sintéticos y los publica en los tópicos correspondientes
2. **Validación:** Un validador consume eventos, valida su esquema, y envía eventos inválidos a una cola de deadletter
3. **Agregación:** Un agregador consume eventos válidos, los deduplica, agrega por ventanas diarias por región, y publica métricas
4. **Auditoría:** Un servicio de auditoría que persiste la trazabilidad de eventos (qué eventos de entrada contribuyeron a qué métricas de salida)
5. **Visualización:** Una API o dashboard mínimo permite consultar métricas agregadas

3.3. Arquitectura Mínima

El sistema debe implementarse siguiendo una arquitectura de microservicios, donde cada servicio principal debe ejecutarse en un contenedor Docker separado. Esta separación permite:

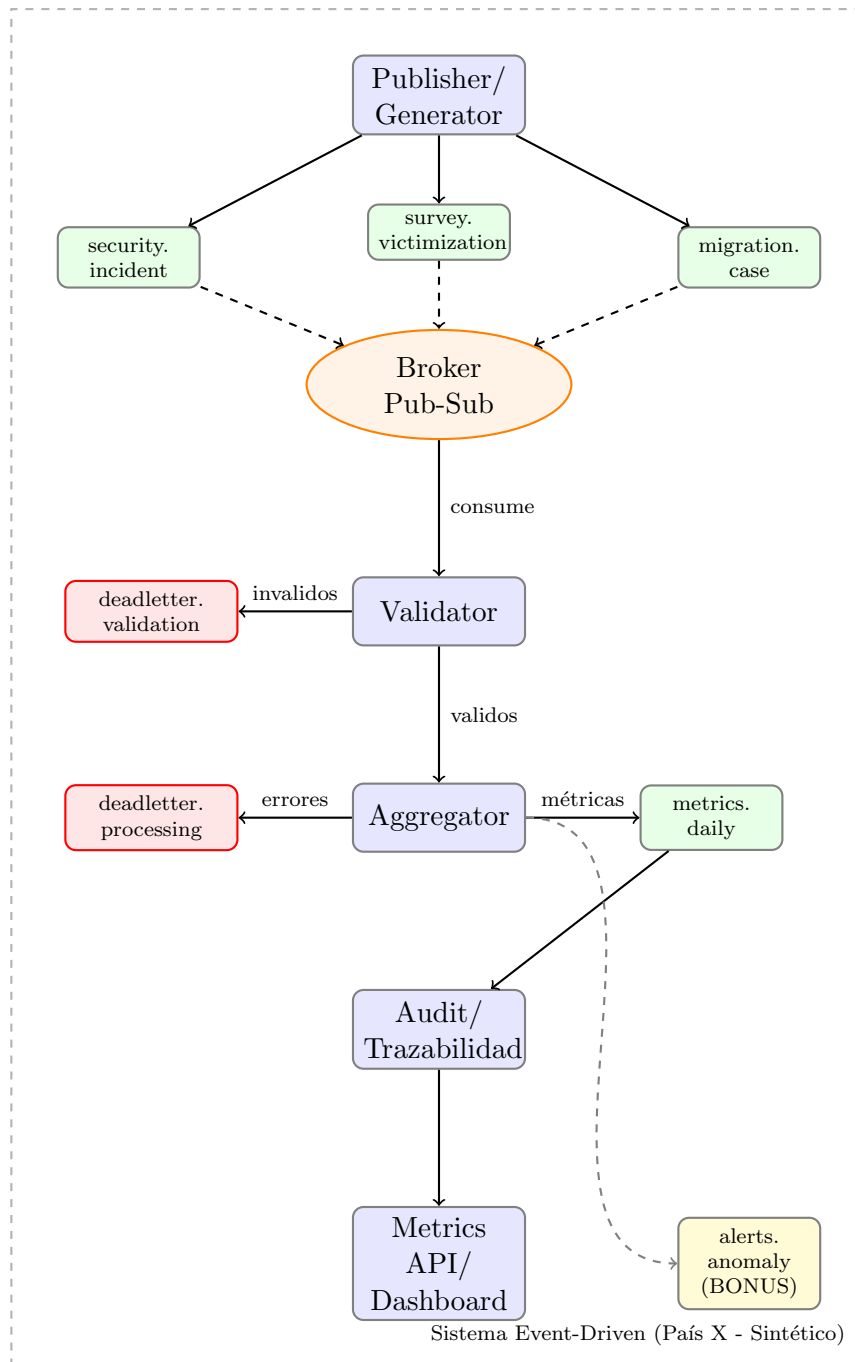
- **Distribución real:** Demostrar que el sistema es verdaderamente distribuido
- **Tolerancia a fallas:** Poder matar servicios individuales para probar recuperación
- **Escalabilidad:** Escalar servicios independientemente según necesidad
- **Aislamiento:** Cada servicio puede fallar sin afectar a los demás

Requisitos de contenedores:

- **Servicios obligatorios separados:** Los siguientes servicios deben estar en contenedores distintos:
 1. Publisher/Generator
 2. Validator
 3. Aggregator
 4. Audit/Trazabilidad
 5. Metrics API/Dashboard
- **Broker pub-sub:** Debe estar en su propio contenedor (Kafka, RabbitMQ, Mosquitto, NATS, Redis según tecnología elegida)
- **Base de datos:** Si se usa PostgreSQL, debe estar en contenedor separado. SQLite puede estar en el mismo contenedor que Audit/Trazabilidad
- **Orquestación:** Todos los servicios deben levantarse con `docker compose up`
- **Comunicación:** Los servicios deben comunicarse a través de la red de Docker (usando nombres de servicio de docker-compose)

Nota: Si un grupo necesita combinar algunos servicios menores por razones técnicas válidas, debe justificarlo en el README. Sin embargo, los 5 servicios principales deben estar separados.

La arquitectura mínima del sistema es:



Leyenda:

- Servicios (microservicios en contenedores)
- Tópicos/Streams del broker
- Deadletter queues
- Broker pub-sub
- Funcionalidad bonus

4. Especificación de Tópicos

4.1. Tópicos Obligatorios

El sistema debe implementar los siguientes tópicos:

1. `security.incident` - Eventos de delitos reportados
2. `survey.victimization` - Eventos de encuestas de victimización
3. `migration.case` - Eventos de casos de migración
4. `metrics.daily` - Métricas agregadas diarias por región
5. `deadletter.validation` - Eventos que fallaron validación de esquema
6. `deadletter.processing` - Eventos que fallaron durante el procesamiento
7. `alerts.anomaly` - (BONUS) Alertas de anomalías detectadas

4.2. Formato de Eventos

Todos los eventos deben seguir el siguiente esquema JSON mínimo:

```
{
  "event_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2025-01-15T10:30:00Z",
  "region": "norte",
  "source": "security.incident",
  "schema_version": "1.0",
  "correlation_id": "corr-12345",
  "payload": {
    // Contenido específico del tipo de evento
  }
}
```

4.2.1. Campos Obligatorios

- `event_id`: UUID v4 único para cada evento
- `timestamp`: ISO-8601 con timezone (UTC)
- `region`: String identificando la región (ej: “norte”, “sur”, “centro”, “este”, “oeste”)
- `source`: Tipo de evento (debe coincidir con el tópico)
- `schema_version`: Versión del esquema (string semver, ej: “1.0”)
- `correlation_id`: ID para correlacionar eventos relacionados (opcional pero recomendado)
- `payload`: Objeto JSON con los datos específicos del evento

4.2.2. Ejemplos de Payloads

security.incident (delitos):

```
{
  "event_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2025-01-15T10:30:00Z",
  "region": "norte",
  "source": "security.incident",
  "schema_version": "1.0",
  "correlation_id": "corr-12345",
  "payload": {
    "crime_type": "theft",
    "severity": "medium",
    "location": {
      "latitude": -33.4489,
      "longitude": -70.6693
    },
    "reported_by": "citizen"
  }
}
```

survey.victimization:

```
{
  "event_id": "660e8400-e29b-41d4-a716-446655440001",
  "timestamp": "2025-01-15T11:00:00Z",
  "region": "sur",
  "source": "survey.victimization",
  "schema_version": "1.0",
  "correlation_id": "corr-12346",
  "payload": {
    "survey_id": "survey-2025-001",
    "respondent_age": 35,
    "victimization_type": "property_crime",
    "incident_date": "2025-01-10",
    "reported": true
  }
}
```

migration.case:

```
{
  "event_id": "770e8400-e29b-41d4-a716-446655440002",
  "timestamp": "2025-01-15T11:30:00Z",
  "region": "centro",
  "source": "migration.case",
}
```

```
"schema_version": "1.0",
"correlation_id": "corr-12347",
"payload": {
  "case_id": "mig-2025-001",
  "case_type": "asylum",
  "status": "pending",
  "origin_country": "country-x",
  "application_date": "2025-01-01"
}
}
```

5. Servicios Mínimos Requeridos

5.1. 1. Publisher/Generator

- **Función:** Genera eventos sintéticos y los publica en los tópicos correspondientes
- **Modos de operación:**
 - **Normal:** Genera eventos a tasa constante configurable
 - **Burst:** Genera picos de eventos (útil para pruebas de backpressure, es decir, cuando el sistema debe regular el flujo porque los consumidores no pueden procesar eventos tan rápido como se publican)
 - **Duplicados controlados:** Inyecta eventos duplicados (mismo `event_id`) para probar deduplicación
 - **Out-of-order:** Publica eventos con timestamps fuera de orden para probar ordenamiento
- **Reproducibilidad:** Debe soportar un parámetro `--seed` para generar eventos reproducibles
- **Configuración:** Tasa de eventos, distribución de regiones, tipos de eventos

5.2. 2. Validator

- **Función:** Consume eventos de los tópicos de entrada, valida el esquema JSON, y publica eventos válidos o los envía a deadletter
- **Validación:**
 - Verificar presencia de campos obligatorios
 - Validar formato de UUID en `event_id`
 - Validar formato ISO-8601 en `timestamp`
 - Validar que `source` coincida con el tópico
 - Validar estructura del `payload` según el tipo de evento

- **Deadletter:** Eventos inválidos deben publicarse en `deadletter.validation` con información del error
- **Idempotencia:** Debe manejar eventos duplicados correctamente

5.3. 3. Aggregator

- **Función:** Consume eventos válidos, los deduplica por `event_id`, agrega por ventanas diarias por región, y publica métricas
- **Deduplicación:** Mantener un registro de `event_id` procesados (ventana deslizando o TTL)
- **Agregación:**
 - Ventanas diarias por región (UTC)
 - Contar eventos por tipo y región
 - Calcular métricas básicas (promedios, totales)
- **Publicación:** Publica métricas agregadas en `metrics.daily` al finalizar cada día
- **Manejo de errores:** Eventos que fallan durante el procesamiento van a `deadletter.processing`

5.4. 4. Audit/Trazabilidad

- **Función:** Persiste la trazabilidad de eventos (qué eventos de entrada contribuyeron a qué métricas de salida)
- **Almacenamiento:** SQLite (desarrollo) o PostgreSQL (producción)
- **Esquema mínimo:**
 - Tabla de eventos de entrada (`event_id`, `timestamp`, `region`, `source`, `payload`)
 - Tabla de métricas de salida (`metric_id`, `date`, `region`, `metrics`)
 - Tabla de trazabilidad (`event_id`, `metric_id`, `contribution_type`)
- **Consultas:** Debe permitir consultar qué eventos contribuyeron a una métrica específica

5.5. 5. Metrics API / Dashboard

- **Función:** Expone métricas agregadas para consulta
- **Implementación mínima:**
 - Endpoint REST `/metrics` que retorna métricas en JSON
 - Filtros por región, fecha, tipo de evento
 - Alternativa: Exportar a archivo o base de datos consultable
- **Formato de respuesta:**

```

{
  "date": "2025-01-15",
  "region": "norte",
  "metrics": {
    "security.incident": {
      "count": 150,
      "by_severity": {
        "low": 50,
        "medium": 80,
        "high": 20
      },
      "by_crime_type": {
        "theft": 60,
        "assault": 40,
        "burglary": 30,
        "other": 20
      }
    },
    "survey.victimization": {
      "count": 200,
      "reported_rate": 0.65
    },
    "migration.case": {
      "count": 75,
      "by_status": {
        "pending": 30,
        "approved": 25,
        "rejected": 20
      }
    }
  }
}

```

6. Requisitos No Negociables

Los siguientes requisitos son obligatorios y deben demostrarse durante la evaluación:

6.1. Checklist de Requisitos Funcionales

RF1: Delivery at-least-once end-to-end: El sistema debe garantizar que cada evento válido se procese al menos una vez, desde la publicación hasta la métrica final

RF2: Idempotencia y deduplicación: El agregador debe detectar y eliminar eventos duplicados (mismo `event_id`) antes de agregar

RF3: Backpressure y retries: El sistema debe implementar:

- **Política de retry configurable:** Cuando un evento falla al ser procesado (por ejemplo, error de validación o fallo temporal del consumidor), el sistema debe reintentar su procesamiento. La política debe ser configurable y puede incluir:
 - **Estrategia de backoff:** Define cómo aumenta el tiempo de espera entre reintentos:
 - **Exponencial:** El tiempo de espera se duplica en cada intento (ej: 1s, 2s, 4s, 8s, 16s). Útil cuando el problema puede resolverse con tiempo y se quiere evitar sobrecargar el sistema.
 - **Lineal:** El tiempo de espera aumenta de forma constante (ej: 1s, 2s, 3s, 4s, 5s). Balance entre rapidez y carga del sistema.
 - **Fijo:** El tiempo de espera es constante entre todos los reintentos (ej: 2s, 2s, 2s, 2s). Más simple pero puede sobrecargar si el problema persiste.
 - **Número máximo de reintentos:** Límite de intentos antes de enviar el evento a deadletter
 - **Intervalo inicial:** Tiempo base de espera antes del primer reintento
- **Manejo de backpressure:** Cuando los consumidores no pueden procesar eventos tan rápido como se publican, el sistema debe detectar esta situación y aplicar mecanismos de regulación:
 - Detectar acumulación de eventos en el broker (lag/backlog)
 - Reducir la tasa de publicación automáticamente o pausar temporalmente
 - Escalar consumidores si es posible (según la tecnología)
 - Monitorear y alertar sobre situaciones de backpressure
- **Límites de reintentos:** Después de alcanzar el número máximo de reintentos configurado, los eventos que siguen fallando deben ser enviados a `deadletter.processing` con información del error y número de intentos realizados

RF4: Tolerancia a fallas: Durante la demo, deben:

- Matar un consumidor (validator o aggregator) y mostrar recuperación automática
- Reiniciar el broker y mostrar que el sistema continúa procesando eventos
- Demostrar que no se pierden eventos durante las fallas

RF5: Replay: Deben demostrar capacidad de re-procesamiento:

- Reprocesar eventos desde un punto en el tiempo específico
- Reprocesar eventos desde un offset/posición específica
- La implementación depende de la tecnología (Kafka: offsets, RabbitMQ: re-queue, etc.)

RF6: Observabilidad mínima:

- Logs estructurados (JSON) con niveles apropiados
- Métricas expuestas: throughput (eventos/segundo), error rate (errores/segundo), lag/backlog (eventos pendientes)

- Las métricas pueden exportarse a archivo, base de datos, o endpoint HTTP

RF7: Reproducibilidad:

- `docker compose up` debe levantar todo el sistema
- Script `make demo` (o equivalente) debe ejecutar una demostración completa
- El generador debe soportar `--seed` para reproducibilidad

RF8: Runtime de demo: La demostración completa debe ejecutarse en ≤ 8 minutos en una máquina estándar

RF9: Datos sintéticos: No usar datos reales ni sensibles; solo datos sintéticos generados

6.2. Requisitos Técnicos

RT1: Contenedores: Los siguientes servicios deben estar en contenedores Docker separados:

- Publisher/Generator (1 contenedor)
- Validator (1 contenedor)
- Aggregator (1 contenedor)
- Audit/Trazabilidad (1 contenedor, puede incluir SQLite si se usa)
- Metrics API/Dashboard (1 contenedor)
- Broker pub-sub (1 contenedor: Kafka, RabbitMQ, Mosquitto, NATS, o Redis según tecnología)
- Base de datos PostgreSQL (1 contenedor, solo si se usa PostgreSQL en lugar de SQLite)

Total mínimo: 6 contenedores (5 servicios + 1 broker). Si se usa PostgreSQL: 7 contenedores.

RT2: Orquestación: Usar `docker-compose.yml` para orquestar todos los servicios. Los servicios deben comunicarse usando los nombres de servicio definidos en `docker-compose` (ej: `validator`, `aggregator`, etc.)

RT3: CI/CD: Repositorio debe tener CI configurado (al menos validación básica: lint, compilación si aplica)

RT4: Documentación: README.md ejecutable con:

- Instrucciones de instalación y ejecución
- Descripción de cada servicio
- Cómo ejecutar la demo
- Cómo ejecutar tests

RT5: Schemas: Definir esquemas JSON Schema o Avro para validación (según tecnología)

RT6: Scripts:

- `run_load.sh` - Ejecuta carga normal de eventos
- `run_burst.sh` - Ejecuta carga con picos
- `run_chaos.sh` - Ejecuta pruebas de caos (matar servicios, reiniciar broker)
- `replay.sh` - Demuestra capacidad de replay

RT7: Tests básicos: Incluir tests unitarios o de integración básicos (según el lenguaje)

7. Cronograma

El laboratorio tiene una duración aproximada de 4 semanas. A continuación se presenta un cronograma sugerido con hitos principales:

7.1. Semana 1: Diseño y Elección de Tecnología

Hitos de la Semana 1:

- **Días 1–3:** Elección de tecnología y diseño de arquitectura
 - Investigar y elegir tecnología pub-sub (comunicar elección al profesor)
 - Revisar documentación de la tecnología elegida
 - Diseñar esquemas de eventos (JSON Schema)
 - Definir estructura de contenedores y docker-compose
 - Asignar servicios a integrantes del grupo
- **Días 4–5:** Configuración inicial y prototipo
 - Configurar entorno de desarrollo (Docker, dependencias)
 - Crear estructura base del proyecto
 - Implementar prototipo básico del Publisher/Generator
 - Configurar docker-compose inicial

7.2. Semana 2: Implementación de Servicios Base

Hitos de la Semana 2:

- **Días 6–8:** Implementación de servicios principales
 - Implementar Publisher/Generator completo con modo normal
 - Implementar Validator con validación de esquemas
 - Implementar Aggregator con deduplicación básica
 - Integrar servicios en docker-compose
- **Días 9–10:** Integración y pruebas iniciales
 - Integrar todos los servicios
 - Probar flujo end-to-end básico
 - Identificar y corregir problemas de integración
 - Implementar tests básicos

7.3. Semana 3: Robustez y Funcionalidades Avanzadas

Hitos de la Semana 3:

- **Días 11–13:** Implementación de requisitos avanzados
 - Implementar backpressure y retries con política definida
 - Implementar tolerancia a fallas (recuperación de consumidores)
 - Implementar replay según tecnología elegida
 - Agregar observabilidad (logs estructurados, métricas)
- **Días 14–15:** Modos especiales y casos edge
 - Implementar modo burst del generador
 - Implementar inyección controlada de duplicados
 - Implementar inyección de eventos out-of-order
 - Implementar soporte de seed para reproducibilidad
 - Probar tolerancia a fallas del broker

7.4. Semana 4: Auditoría, Visualización y Entrega

Hitos de la Semana 4:

- **Días 16–18:** Servicios complementarios y scripts
 - Implementar servicio de Audit/Trazabilidad
 - Implementar Metrics API o Dashboard
 - Crear scripts de demo (run_load, run_burst, run_chaos, replay)
 - Implementar bonus de alertas de anomalías (opcional)
- **Días 19–21:** Preparación de entrega
 - Completar y revisar README.md
 - Preparar y practicar guion de demo (máximo 8 minutos)
 - Revisar y probar todos los entregables
 - Configurar CI/CD si aún no está hecho
 - Realizar pruebas finales de reproducibilidad

Nota: Este cronograma es una guía sugerida. Los grupos pueden ajustar el ritmo según sus necesidades, pero deben asegurarse de cumplir con todos los requisitos antes de la fecha de entrega.

8. Entregables

8.1. Repositorio Git

El repositorio debe contener:

- **docker-compose.yml** - Orquestación de todos los servicios
- **README.md** - Documentación ejecutable con instrucciones completas
- **Schemas/** - Esquemas JSON Schema o Avro para validación
- **Scripts/** - Scripts de ejecución:
 - `run_load.sh` (o equivalente)
 - `run_burst.sh`
 - `run_chaos.sh`
 - `replay.sh`
- **Servicios/** - Código fuente de cada microservicio organizado por servicio
- **tests/** - Tests básicos (unitarios o de integración)
- **.github/workflows/** o **.gitlab-ci.yml** - Configuración de CI
- **Makefile** (opcional pero recomendado) - Comandos comunes como `make demo`

8.2. Demo

Demo de máximo 8 minutos con guion obligatorio que incluya:

1. **Ingestión (1.5 min):** Mostrar generador publicando eventos en modo normal
2. **Validación (1 min):** Mostrar validator procesando eventos y enviando inválidos a deadletter
3. **Métricas (1.5 min):** Mostrar agregador generando métricas diarias y consulta en API/Dashboard
4. **Falla inducida (1.5 min):** Matar un consumidor y mostrar recuperación automática
5. **Recuperación (1 min):** Reiniciar broker y mostrar continuidad del procesamiento
6. **Replay (1.5 min):** Demostrar re-procesamiento desde un punto específico

Total: máximo 8 minutos. El guion debe estar documentado en el README o en un archivo separado.

9. Rúbrica de Evaluación

La evaluación se realizará comparando las implementaciones entre grupos (mismo problema, diferentes tecnologías). La rúbrica es:

9.1. Correctitud Funcional (25 %)

- **Esquemas de eventos:** Validación correcta de esquemas JSON (10 %)
- **Ventanas de agregación:** Agregación correcta por ventanas diarias por región (10 %)
- **Métricas:** Cálculo correcto de métricas agregadas (5 %)

9.2. Robustez (25 %)

- **Deadletter queues:** Manejo correcto de eventos inválidos y fallidos (5 %)
- **Retry y backpressure:** Política de retries implementada y funcionando (5 %)
- **Tolerancia a fallas:** Recuperación automática ante fallas de consumidores y broker (10 %)
- **Idempotencia:** Deduplicación correcta de eventos duplicados (5 %)

9.3. Distribución Real (20 %)

- **Paralelismo:** Uso de consumer groups o equivalentes para paralelizar procesamiento (5 %)
- **Escalabilidad:** Capacidad de escalar consumidores horizontalmente (5 %)
- **Consumer groups:** Configuración correcta de grupos de consumidores (5 %)
- **Distribución:** Arquitectura verdaderamente distribuida (no solo localhost) (5 %)

9.4. Observabilidad (15 %)

- **Logs estructurados:** Logs en formato JSON con información relevante (5 %)
- **Métricas:** Throughput, error rate, lag/backlog expuestos (5 %)
- **Monitoreo:** Capacidad de monitorear el estado del sistema (5 %)

9.5. Reproducibilidad (15 %)

- **Docker Compose:** `docker compose up` funciona correctamente (5 %)
- **Scripts:** Scripts de demo funcionan correctamente (5 %)
- **README:** Documentación clara y ejecutable (3 %)
- **Seed:** Reproducibilidad mediante seed funciona (2 %)

9.6. Bonus: Alertas de Anomalías (+10 %)

Implementación del tópico `alerts.anomaly` con:

- Detección de anomalías en los datos (ej: picos inusuales, patrones anómalos)
- Publicación de alertas en el tópico correspondiente
- Explicación de la estrategia de detección en el README o durante la demo

10. Errores Comunes a Evitar

10.1. Errores de Diseño

- **No implementar at-least-once:** Asegurarse de que los consumidores confirmen (ack) mensajes solo después de procesarlos completamente
- **Olvidar deduplicación:** El agregador debe mantener un registro de `event_id` procesados
- **No manejar out-of-order:** Si la tecnología lo permite, considerar ordenamiento por timestamp antes de agregar
- **Deadletter sin información:** Los eventos en deadletter deben incluir el error que causó el fallo

10.2. Errores Técnicos

- **No usar consumer groups:** Asegurarse de configurar grupos de consumidores para paralelismo
- **No configurar retries:** Implementar política de retries con límites para evitar loops infinitos
- **No exponer métricas:** El sistema debe permitir monitorear throughput, error rate, y lag
- **Docker compose incompleto:** Todos los servicios deben levantarse con un solo comando

10.3. Errores de Demo

- **Demo demasiado larga:** Respetar el tiempo máximo de 8 minutos
- **No mostrar recuperación:** La demo debe incluir fallas inducidas y recuperación
- **No demostrar replay:** Mostrar claramente la capacidad de re-procesamiento
- **Falta de guion:** Tener un guion claro y practicado antes de la demo

10.4. Errores de Documentación

- **README incompleto:** Debe incluir instrucciones paso a paso para ejecutar el sistema
- **Falta de schemas:** Documentar los esquemas de eventos claramente

11. Recursos y Referencias

11.1. Documentación de Tecnologías

- **Kafka:** <https://kafka.apache.org/documentation/>
- **Redpanda:** <https://docs.redpanda.com/>
- **RabbitMQ:** <https://www.rabbitmq.com/documentation.html>
- **MQTT (Mosquitto):** <https://mosquitto.org/documentation/>
- **NATS JetStream:** <https://docs.nats.io/nats-concepts/jetstream>
- **Redis Streams:** <https://redis.io/docs/data-types/streams/>

11.2. Conceptos Teóricos

- Semánticas de entrega: at-least-once, at-most-once, exactly-once
- Idempotencia y deduplicación en sistemas distribuidos
- Backpressure y rate limiting
- Consumer groups y particionamiento
- Deadletter queues y manejo de errores
- Observabilidad en sistemas distribuidos (logs, métricas, traces)

11.3. Herramientas Útiles

- **Docker & Docker Compose:** <https://docs.docker.com/>
- **JSON Schema:** <https://json-schema.org/>
- **Python clients:** kafka-python, pika (RabbitMQ), paho-mqtt, nats-py, redis-py
- **Go clients:** sarama (Kafka), amqp091-go (RabbitMQ), paho.mqtt.golang, nats.go, go-redis
- **Java clients:** Librerías oficiales de cada tecnología

12. Fecha de Entrega

Fecha límite: 30 de enero de 2026, 23:59 (hora de Chile)

Formato de entrega:

- Repositorio Git con acceso para el profesor
- Tag o commit específico marcado como entrega final
- Demo programada con el profesor (presencial o remota)

13. Notas Finales

- Este laboratorio está diseñado para que todos los grupos resuelvan el mismo problema, permitiendo comparación entre tecnologías
- La evaluación considera tanto la correctitud funcional como la robustez y la capacidad de distribución real
- Se espera que los estudiantes investiguen las capacidades específicas de su tecnología elegida
- El bonus de alertas de anomalías es opcional pero recomendado para grupos que busquen destacar
- Ante dudas, consultar con el profesor durante las horas de oficina o por correo electrónico